解决方案验证和使用情形
NetApp Solutions

NetApp
May 28, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/containers/rh-os-n_use_case_pipeline.html on May 28, 2022. Always check docs.netapp.com for the latest.

目录

解决方案验证和使用情形：采用 NetApp 的 Red Hat OpenShift 1
部署采用永久性存储的 Jenkins CI/CD 管道：采用 NetApp 的 Red Hat OpenShift 1
使用 NetApp ONTAP 在 Red Hat OpenShift 上配置多租户 11

解决方案验证和使用情形：采用 NetApp 的 Red Hat OpenShift

此页面上提供的示例包括解决方案验证以及采用 NetApp 的 Red Hat OpenShift 的用例。

- "部署具有永久性存储的 Jenkins CI/CD 管道"
- "在使用 NetApp 的 Red Hat OpenShift 上配置多租户"
- "借助 NetApp ONTAP 实现 Red Hat OpenShift 虚拟化"
- "借助 NetApp 在 Red Hat OpenShift 上为 Kubernetes 提供高级集群管理"

"接下来：视频和演示。"

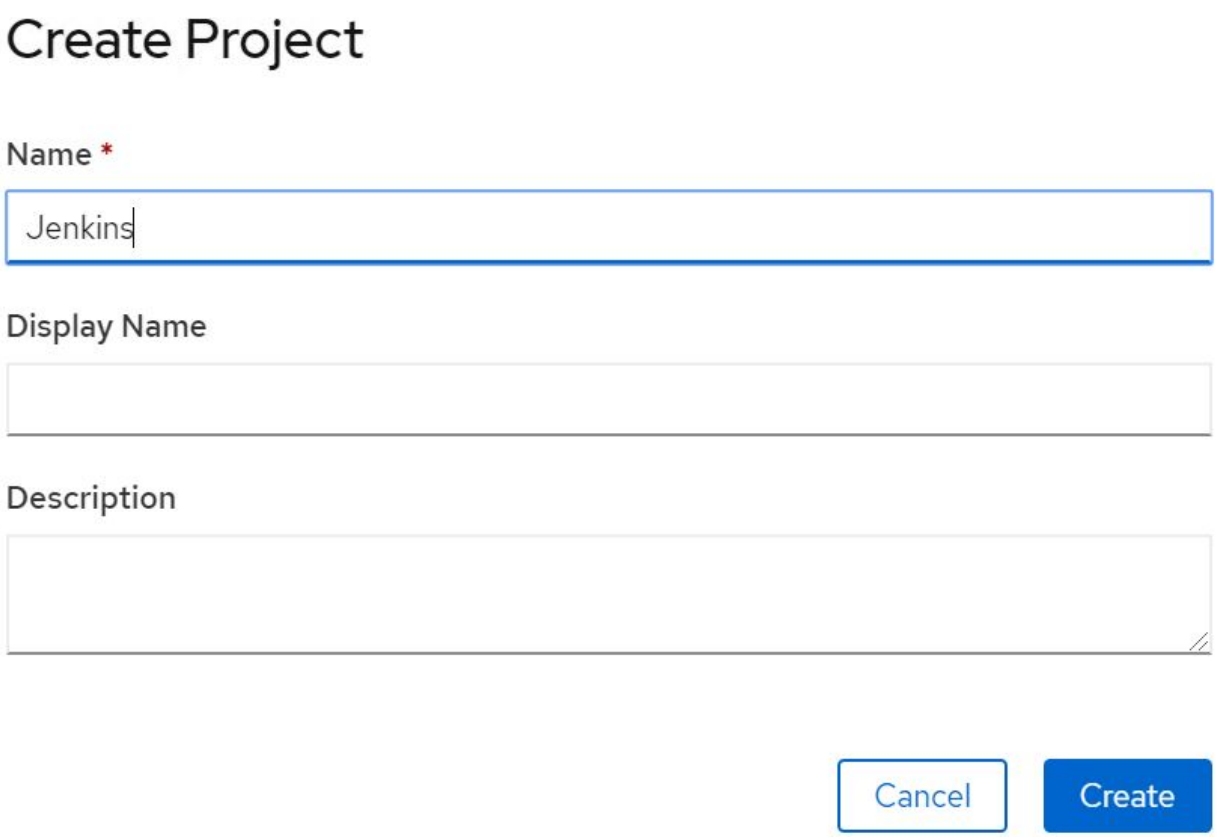
部署采用永久性存储的 Jenkins CI/CD 管道：采用 NetApp 的 Red Hat OpenShift

本节介绍了与 Jenkins 部署持续集成 / 持续交付或部署（ CI/CD ）管道以验证解决方案运行的步骤。

创建 Jenkins 部署所需的资源

要创建部署 Jenkins 应用程序所需的资源，请完成以下步骤：

1. 创建一个名为 Jenkins 的新项目。



2. 在此示例中，我们使用永久性存储部署了 Jenkins 。要支持 Jenkins 构建，请创建 PVC 。导航到 "Storage">"Persistent Volume Claim " ，然后单击 "Create Persistent Volume Claim " 。选择已创建的存储类，确保永久性卷声明名称是 Jenkins ，选择适当的大小和访问模式，然后单击创建。

Project: jenkins

Create Persistent Volume Claim

Edit YAML

Storage Class

SC basic

Storage class for the new claim.

Persistent Volume Claim Name *

jenkins

A unique name for the storage claim within the project.

Access Mode *

Single User (RWO) Shared Access (RWX) Read Only (ROX)

Permissions to the mounted drive.

Size *

100 GiB

Desired storage capacity.

Use label selectors to request storage

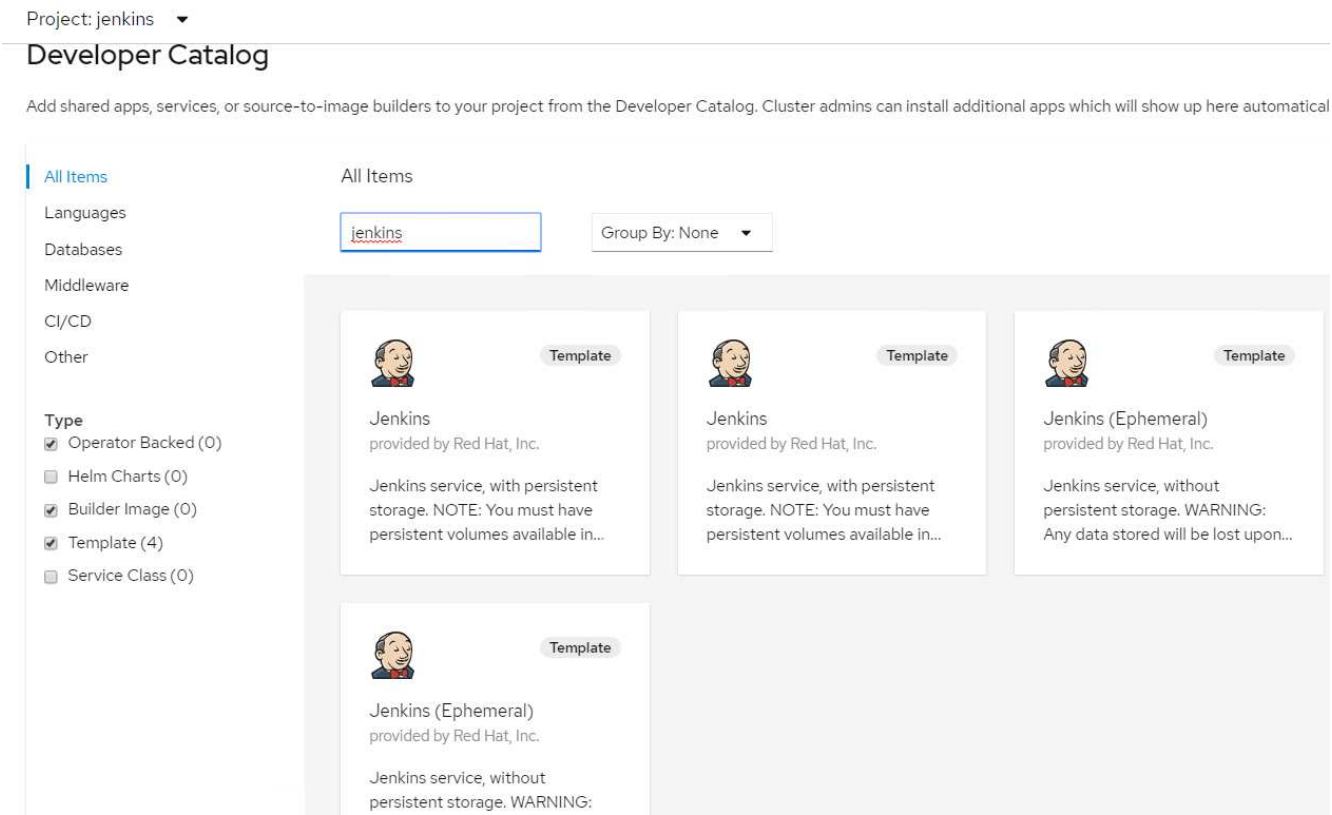
Use label selectors to define how storage is created.

Create Cancel

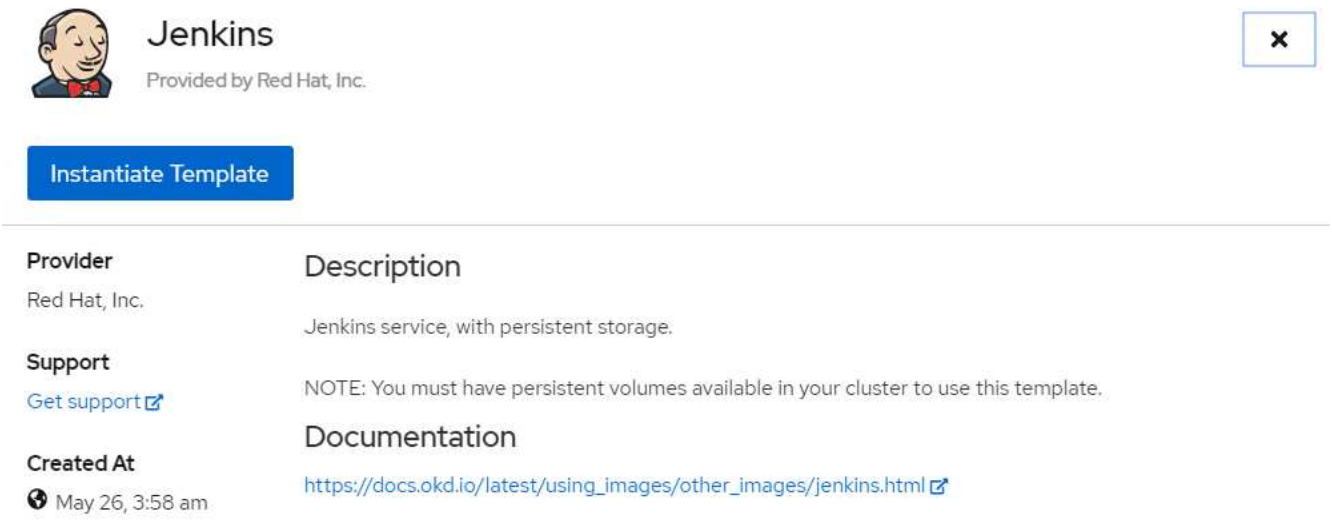
使用永久性存储部署 Jenkins

要使用永久性存储部署 Jenkins ，请完成以下步骤：

1. 在左上角，将角色从管理员更改为开发人员。单击 +Add ，然后从目录中选择。在 Filter by Keyword 栏中，搜索 Jenkins 。选择 Jenkins Service with Persistent Storage 。



2. 单击 实例化模板 。



3. 默认情况下，系统会填充 Jenkins 应用程序的详细信息。根据您的要求，修改参数并单击创建。此过程将创建支持 OpenShift 上的 Jenkins 所需的所有资源。

Instantiate Template

Namespace *

PR jenkins

Jenkins Service Name

jenkins

The name of the OpenShift Service exposed for the Jenkins container.

Jenkins JNLP Service Name

jenkins-jnlp

The name of the service used for master/slave communication.

Enable OAuth in Jenkins

true

Whether to enable OAuth OpenShift integration. If false, the static account 'admin' will be initialized with the password 'password'.

Memory Limit

1Gi

Maximum amount of memory the container can use.

Volume Capacity *

50Gi

Volume space available for data, e.g. 512Mi, 2Gi.

Jenkins ImageStream Namespace

openshift

The OpenShift Namespace where the Jenkins ImageStream resides.

Disable memory intensive administrative monitors

false

Whether to perform memory intensive, possibly slow, synchronization with the Jenkins Update Center on start. If true, the Jenkins core update monitor and site warnings monitor are disabled.

Jenkins ImageStreamTag

jenkins:2

Name of the ImageStreamTag to be used for the Jenkins image.

Fatal Error Log File

false

When a fatal error occurs, an error log is created with information and the state obtained at the time of the fatal error.

Allows use of Jenkins Update Center repository with invalid SSL certificate

false

Whether to allow use of a Jenkins Update Center that uses invalid certificate (self-signed, unknown CA). If any value other than 'false', certificate check is bypassed. By default, certificate check is enforced.

Create Cancel

Jenkins

INSTANT-APP JENKINS

View documentation Get support

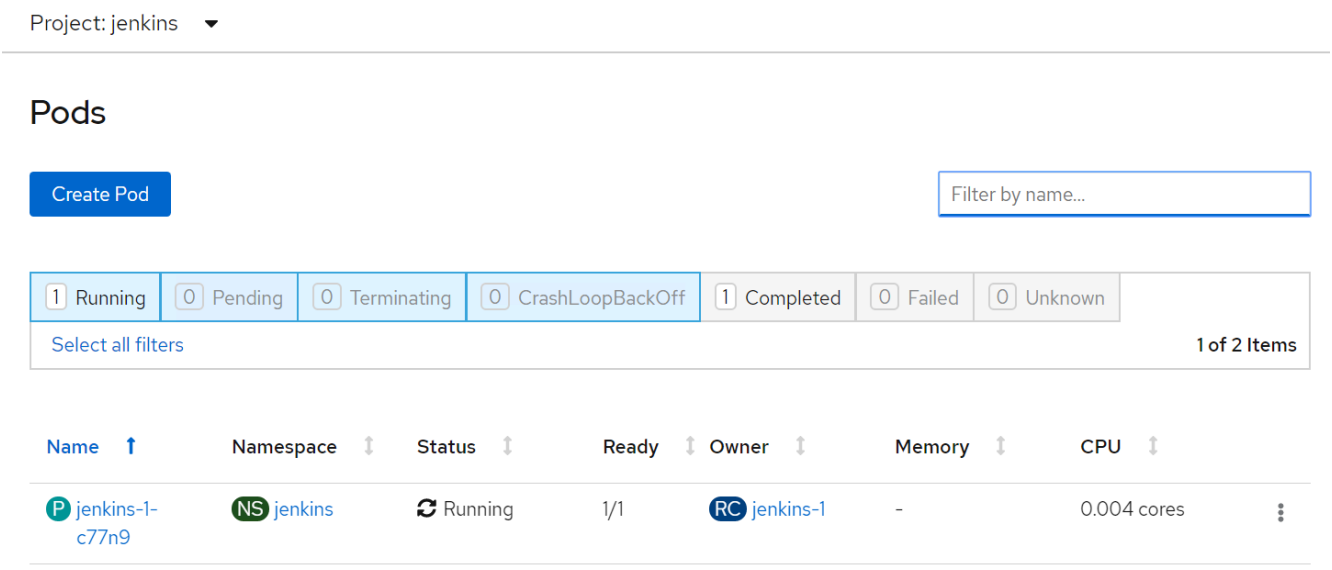
Jenkins service, with persistent storage.

NOTE: You must have persistent volumes available in your cluster to use this template.

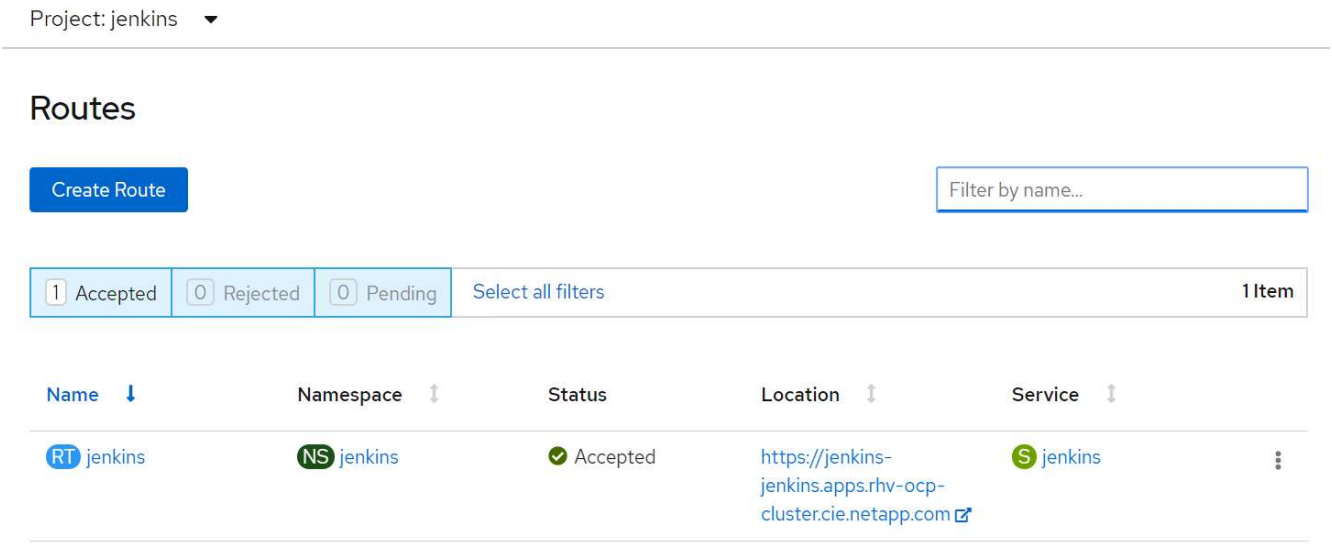
The following resources will be created:

- DeploymentConfig
- PersistentVolumeClaim
- RoleBinding
- Route
- Service
- ServiceAccount

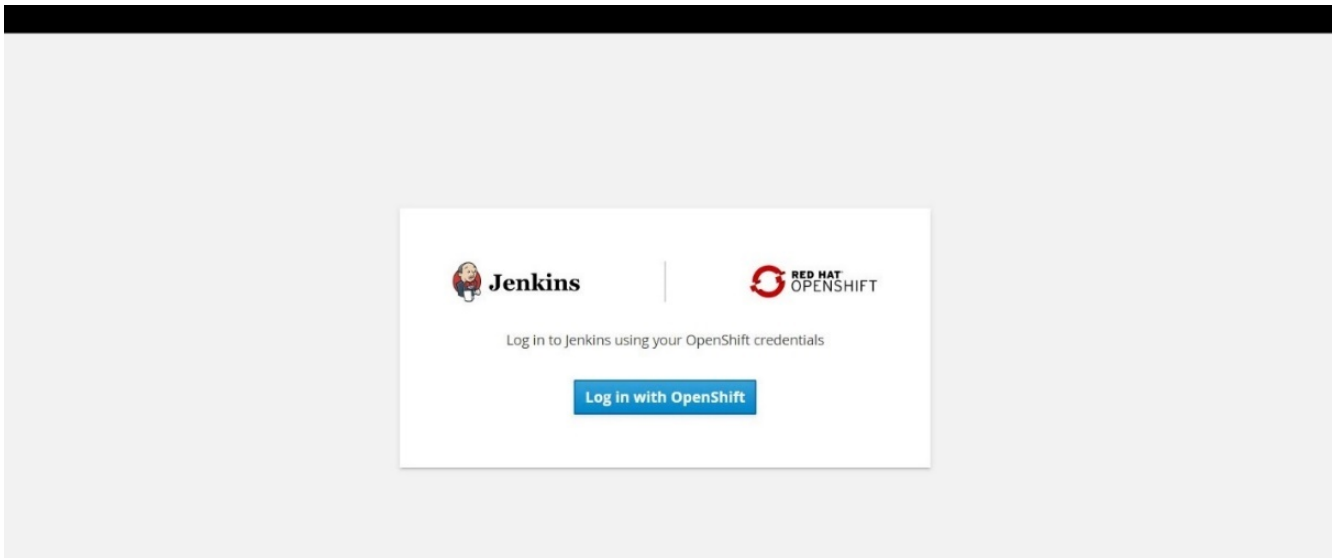
4. Jenkins Pod 大约需要 10 到 12 分钟才能进入就绪状态。



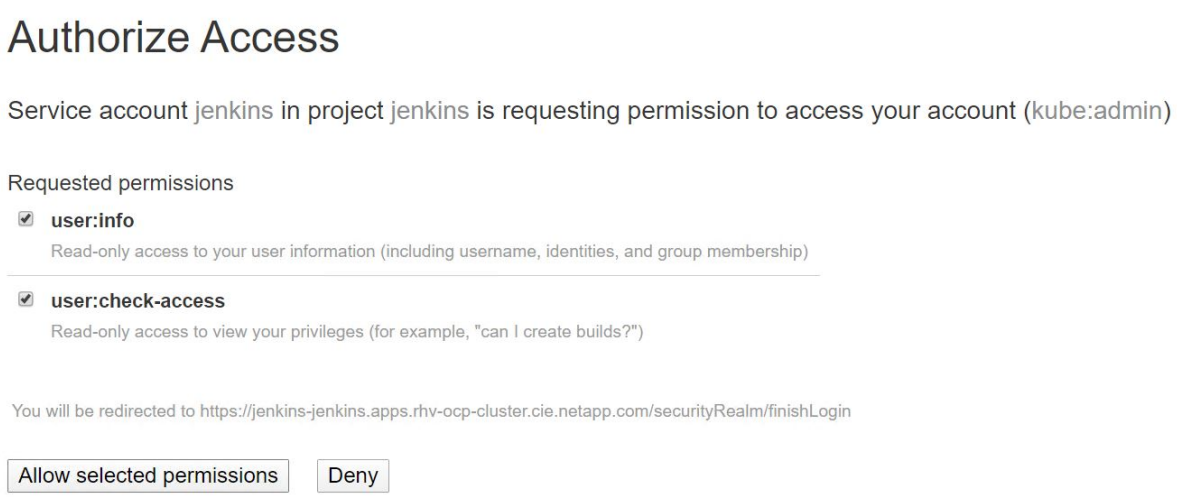
5. 实例化 Pod 后，导航到 "Networking" （网络） >"routes" （路由）。要打开 Jenkins 网页，请单击为 Jenkins 路由提供的 URL 。



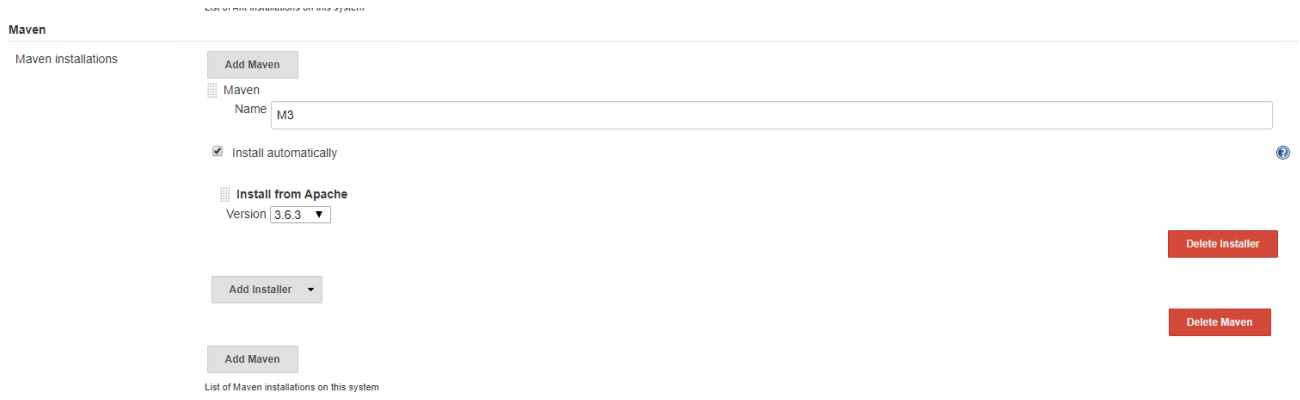
6. 由于在创建 Jenkins 应用程序时使用了 OpenShift OAuth ，因此请单击使用 OpenShift 登录。



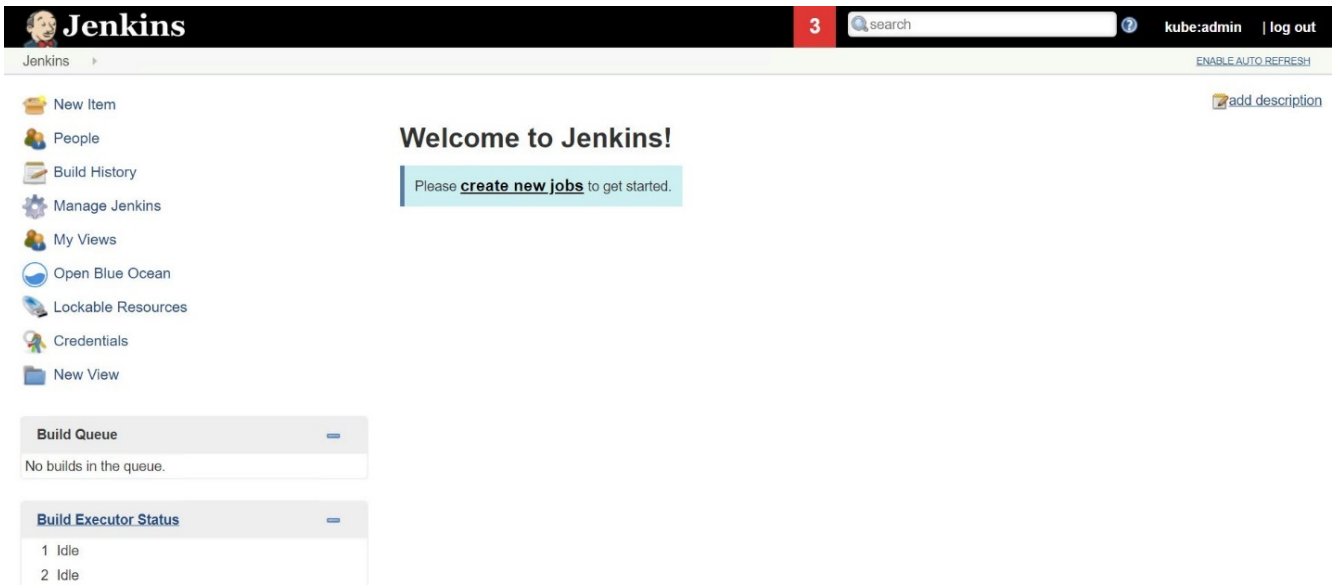
7. 授权 Jenkins 服务帐户访问 OpenShift 用户。



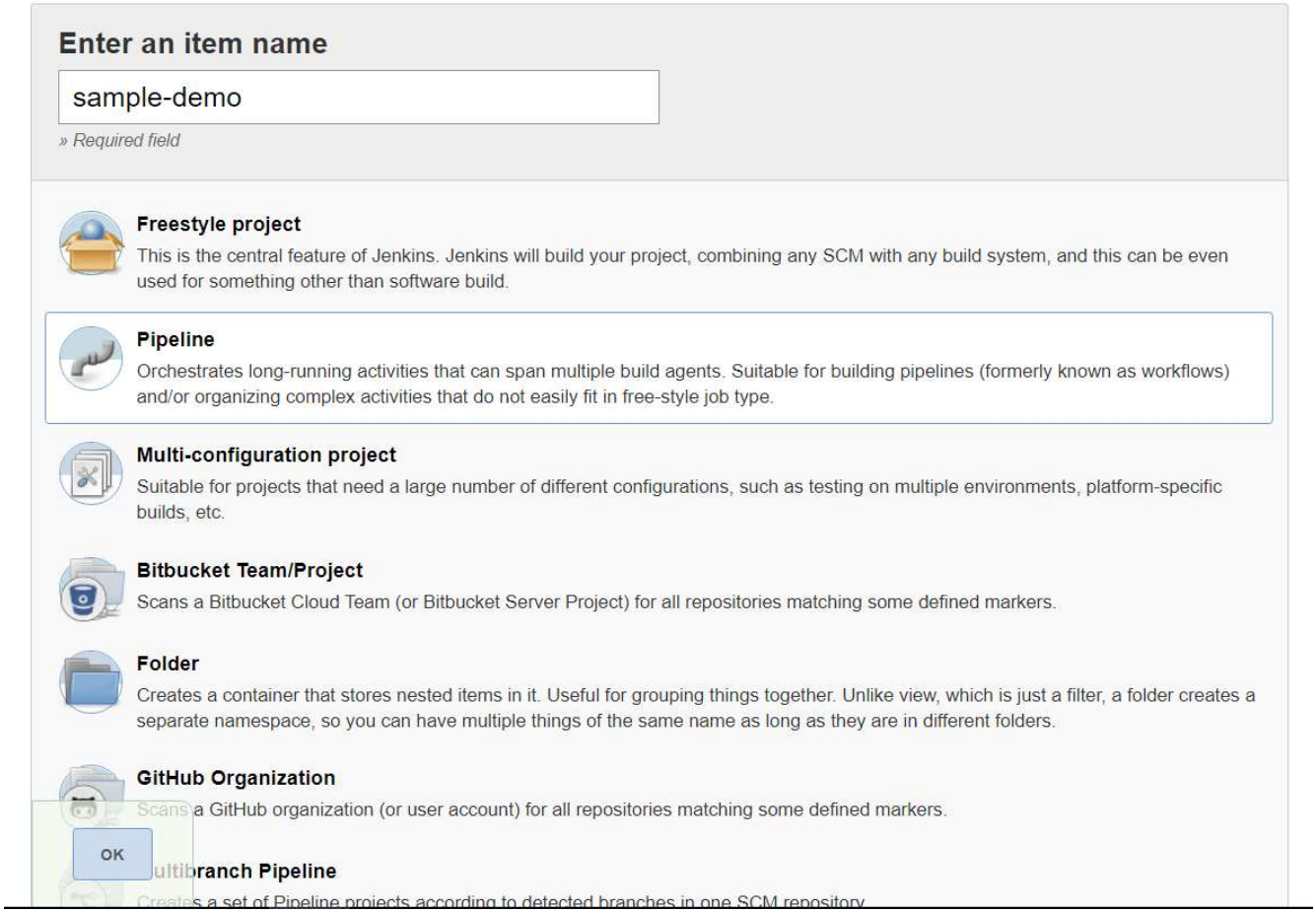
8. 此时将显示 Jenkins 欢迎页面。由于我们使用的是 Maven 内部版本，因此请先完成 Maven 安装。导航到 Manage Jenkins > Global Tool Configuration ，然后在 Maven 子标题中单击 Add Maven 。输入您选择的名称，并确保已选中自动安装选项。单击保存。



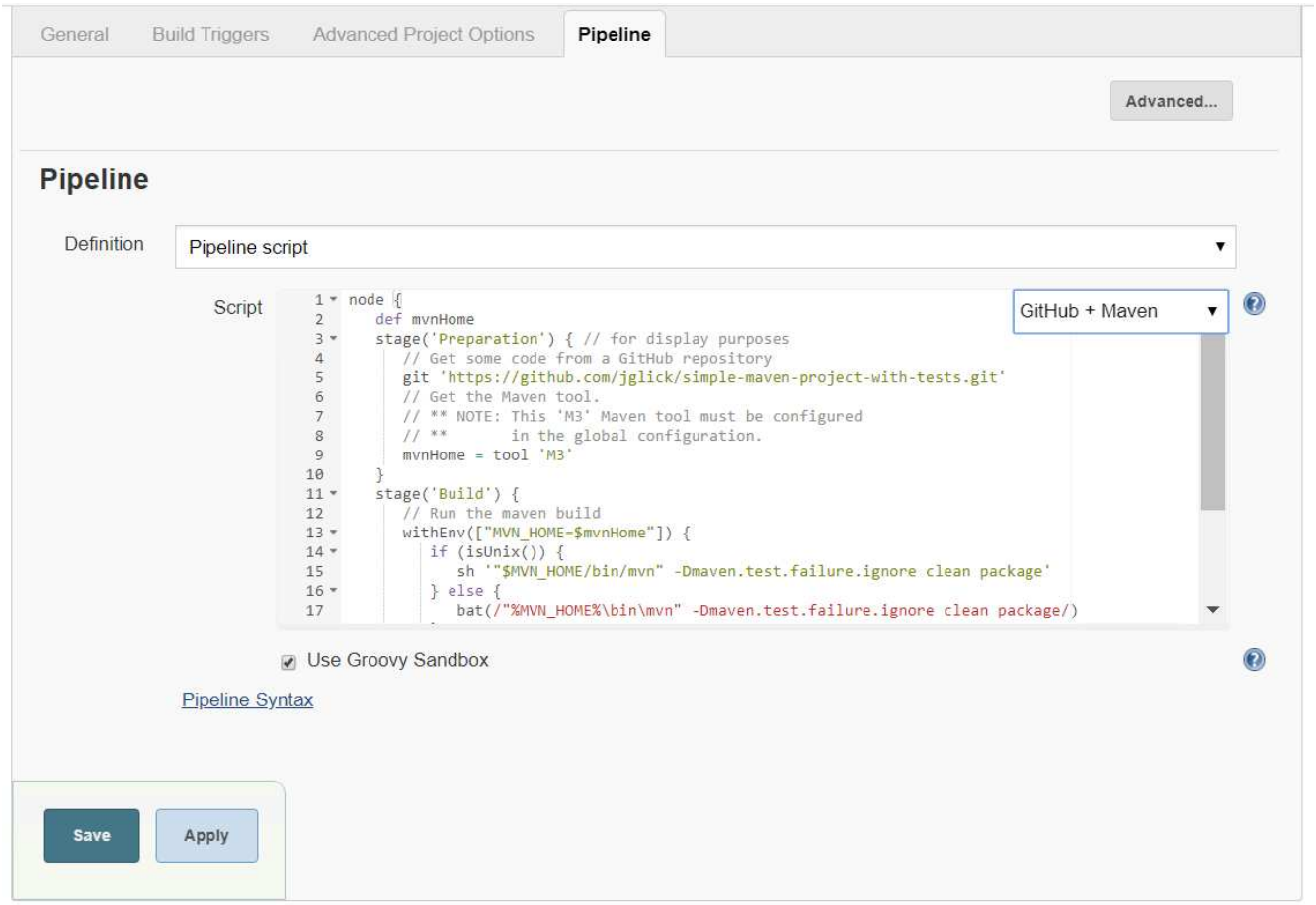
9. 现在，您可以创建一个管道来演示 CI/CD 工作流。在主页上，单击左侧菜单中的创建新作业或新建项目。



10. 在创建项目页面上，输入所选名称，选择管道，然后单击确定。



11. 选择管道选项卡。从试用样本管道下拉菜单中，选择 Github + Maven 。代码将自动填充。单击保存。



12. 单击 Build now ，在准备，构建和测试阶段触发开发。完成整个构建过程并显示构建结果可能需要几分钟的时间。

13. 只要代码发生任何更改，就可以重新构建管道来修补新版本的软件，从而实现持续集成和持续交付。单击 Recent Changes 以跟踪与先前版本相比的更改。

"接下来：视频和演示。"

使用 NetApp ONTAP 在 Red Hat OpenShift 上配置多租户

在使用 NetApp 的 Red Hat OpenShift 上配置多租户

许多在容器上运行多个应用程序或工作负载的组织往往会为每个应用程序或工作负载部署一个 Red Hat OpenShift 集群。这样，他们就可以对应用程序或工作负载实施严格的隔离，优化性能并减少安全漏洞。但是，为每个应用程序部署一个单独的 Red Hat OpenShift 集群会产生自己的一系列问题。它增加了单独监控和管理每个集群所需的运营开销，由于为不同应用程序配置了专用资源而增加了成本，并妨碍了高效的可扩展性。

要解决这些问题，可以考虑在一个 Red Hat OpenShift 集群中运行所有应用程序或工作负载。但是，在这种架构中，资源隔离和应用程序安全漏洞是主要挑战之一。一个工作负载中的任何安全漏洞都可能自然溢出到另一个工作负载，从而增加影响区域。此外，一个应用程序的任何突然不受控制的资源利用率都会影响另一个应用程序的性能，因为默认情况下没有资源分配策略。

因此，企业需要寻找在这两种环境中都能获得最佳性能的解决方案，例如，允许他们在一个集群中运行所有工作负载，同时为每个工作负载提供专用集群的优势。

其中一个有效的解决方案是在 Red Hat OpenShift 上配置多租户。多租户是一种架构，允许多个租户在同一集群上共存，并正确隔离资源，安全性等。在这种情况下，可以将租户视为集群资源的一部分，这些资源配置为供特

定用户组专用使用。在 Red Hat OpenShift 集群上配置多租户具有以下优势：

- 通过共享集群资源，降低资本支出和运营支出
- 降低运营和管理开销
- 保护工作负载免受安全违规交叉影响
- 保护工作负载，防止因资源争用而导致性能意外下降

对于完全实现的多租户 OpenShift 集群，必须为属于不同资源分段的集群资源配置配额和限制：计算，存储，网络连接，安全性等。虽然我们会介绍此解决方案中所有资源分段的某些方面， 我们重点介绍最佳实践，通过在由 NetApp ONTAP 提供支持的 Astra Trident 动态分配的存储资源上配置多租户，隔离并保护同一 Red Hat OpenShift 集群上多个工作负载提供或使用的数据。

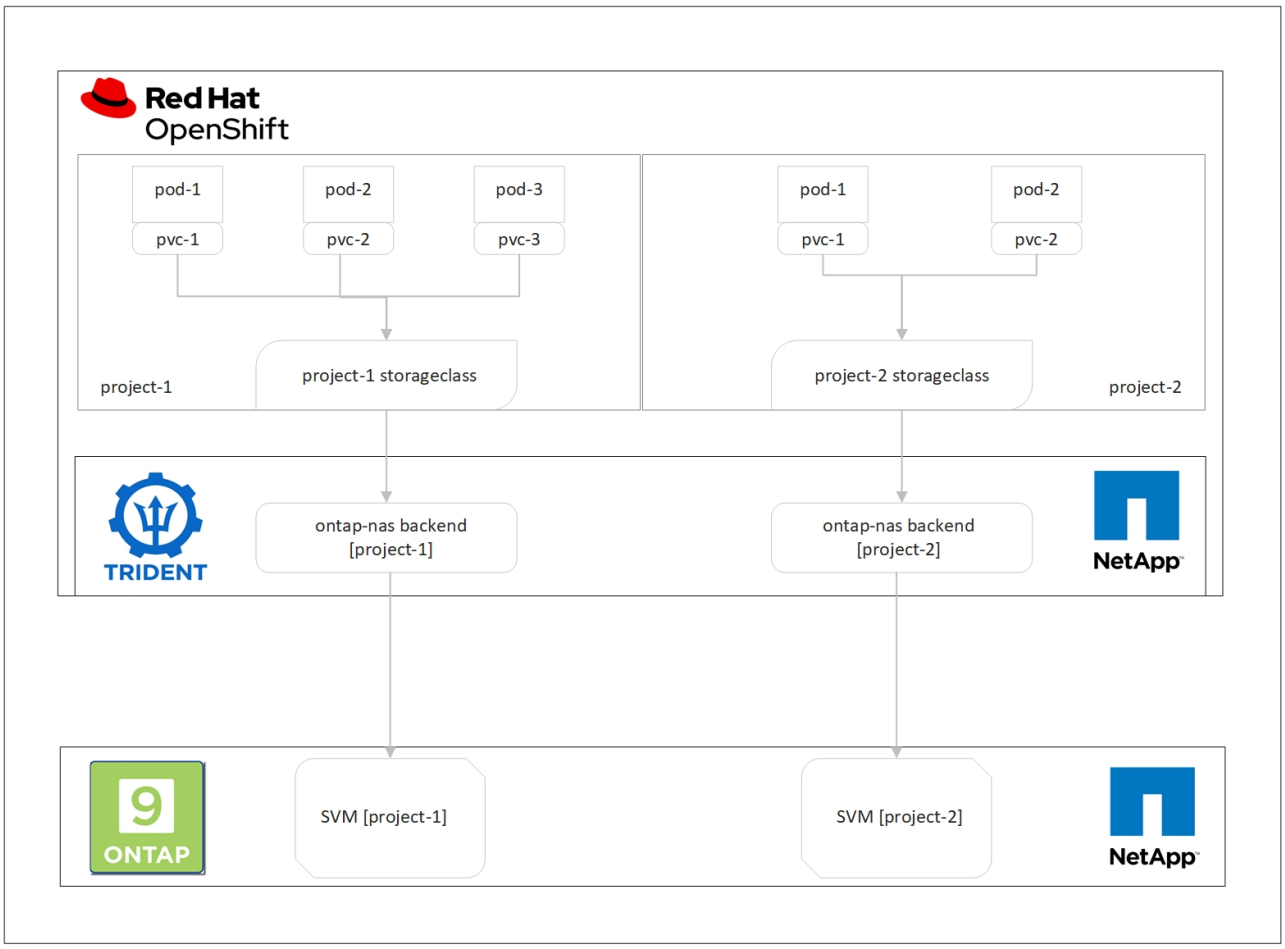
"接下来：架构。"

架构

虽然默认情况下，由 NetApp ONTAP 提供支持的 Red Hat OpenShift 和 Astra Trident 不会在工作负载之间提供隔离，但它们提供了广泛的功能，可用于配置多租户。为了更好地了解如何在采用 NetApp ONTAP 支持的 Astra Trident 的 Red Hat OpenShift 集群上设计多租户解决方案，让我们考虑一个包含一系列要求的示例，并概述其配置。

假设一个组织在一个 Red Hat OpenShift 集群上运行两个工作负载，作为两个不同团队正在处理的两个项目的一部分。这些工作负载的数据驻留在由 NetApp ONTAP NAS 后端的 Astra Trident 动态配置的 PVC 上。该组织需要为这两个工作负载设计多租户解决方案，并隔离用于这些项目的资源，以确保保持安全性和性能，主要侧重于为这些应用程序提供服务的数据。

下图展示了由 NetApp ONTAP 提供支持的带有 Astra Trident 的 Red Hat OpenShift 集群上的多租户解决方案。



技术要求

1. NetApp ONTAP 存储集群
2. Red Hat OpenShift 集群
3. Astra Trident

Red Hat OpenShift —集群资源

从 Red Hat OpenShift 集群的角度来看，要开始使用的顶级资源是项目。OpenShift 项目可以视为将整个 OpenShift 集群划分为多个虚拟集群的集群资源。因此，项目级别的隔离为配置多租户提供了基础。

下一步是在集群中配置 RBAC 。最佳做法是，将处理单个项目或工作负载的所有开发人员配置到身份提供程序（ IdP ）中的单个用户组中。Red Hat OpenShift 允许 IdP 集成和用户组同步，从而允许将 IdP 中的用户和组导入到集群中。这样可以帮助集群管理员将项目专用集群资源的访问权限隔离给一个或多个处理该项目的用户组，从而限制对任何集群资源的未授权访问。要了解有关 IdP 与 Red Hat OpenShift 集成的详细信息，请参见相关文档 "此处"。

NetApp ONTAP

必须隔离用作 Red Hat OpenShift 集群永久性存储提供程序的共享存储，以确保在存储上为每个项目创建的卷在主机上显示为它们，就像在单独的存储上创建一样。为此，请在 NetApp ONTAP 上创建与项目或工作负载数量相同的 SVM （ Storage Virtual Machine ），并将每个 SVM 专用于一个工作负载。

Astra Trident

在 NetApp ONTAP 上为不同项目创建不同的 SVM 之后，必须将每个 SVM 映射到不同的 Trident 后端。Trident 上的后端配置会将永久性存储分配给 OpenShift 集群资源，并且需要将 SVM 的详细信息映射到。此驱动程序至少应为后端的协议驱动程序。或者，您也可以通过它定义如何在存储上配置卷，并设置卷大小或聚合使用量等限制。有关 Trident 后端定义的详细信息，请参见 "此处"。

Red Hat OpenShift —存储资源

配置 Trident 后端后，下一步是配置 StorageClasses 。配置与后端相同数量的存储类，为每个存储类提供访问权限，以便仅在一个后端启动卷。在定义存储类时，我们可以使用 storagePools 参数将 StorageClass 映射到特定的 Trident 后端。可以找到用于定义存储类的详细信息 "此处"。因此，从 StorageClass 到 Trident 后端存在一对一映射，这种映射可指向一个 SVM 。这样可以确保通过分配给该项目的 StorageClass 处理的所有存储请求仅由专用于该项目的 SVM 处理。

由于存储类不是命名空间资源，我们如何确保拒绝另一命名空间或项目中的 Pod 向一个项目的存储类声明？问题解答将使用 ResourceQuotas 。ResourceQuotas 是控制每个项目资源总使用量的对象。它可以限制项目中的对象可以使用的资源数量以及总资源量。使用 ResourceQuotas 几乎可以限制项目中的所有资源，而高效地使用此功能可以帮助组织降低因过度配置或过度消耗资源而导致的成本和中断。请参见文档 "此处" 有关详细信息 …

对于这种使用情形，我们需要限制特定项目中的 Pod 从非专用于其项目的存储类中申请存储。为此，我们需要通过将 ` <storage-class-name>.storageclass.storage.k8s.io/persistentvolumeclaims` 设置为 0 来限制其他存储类的永久性卷请求。此外，集群管理员必须确保项目中的开发人员不应有权修改 ResourceQuotas 。

"下一步：配置。"

Configuration

对于任何多租户解决方案，任何用户都无法访问比所需更多的集群资源。因此，要在多租户配置中配置的整个资源集将在集群管理员，存储管理员和处理每个项目的开发人员之间进行划分。

下表概括了不同用户要执行的不同任务：

Role	任务
* 集群管理 *	为不同的应用程序或工作负载创建项目
	为 storage-admin 创建 ClusterRoles 和 RoleBindings
	为分配对特定项目的访问权限的开发人员创建角色和角色绑定
	[可选] 配置项目以在特定节点上计划 Pod
* 存储管理 *	在 NetApp ONTAP 上创建 SVM
	创建 Trident 后端
	创建 StorageClasses
	创建存储 ResourceQuotas

Role	任务
* 开发人员 *	验证对已分配项目中的 PVC 或 Pod 的创建或修补访问权限
	验证对在其他项目中创建或修补 PVC 或 Pod 的访问权限
	验证对查看或编辑项目， ResourceQuotas 和 StorageClasses 的访问权限

"下一步：前提条件。"

Configuration

前提条件

- NetApp ONTAP 集群。
- Red Hat OpenShift 集群
- 集群上安装的 Trident 。
- 安装了 tridentctl 和 oc 工具并将其添加到 $path 中的管理工作站。
- 对 ONTAP 的管理员访问权限。
- 对 OpenShift 集群的集群管理员访问。
- 集群已与身份提供程序集成。
- 身份提供程序经过配置，可以有效区分不同团队中的用户。

"下一步：集群管理员任务。"

配置： **cluster-admin** 任务

Red Hat OpenShift cluster-admin 执行以下任务：

1. 以 cluster-admin 身份登录到 Red Hat OpenShift 集群。
2. 创建两个与不同项目对应的项目。

```
oc create namespace project-1
oc create namespace project-2
```

3. 为 project-1 创建开发人员角色。

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-1
  name: developer-project-1
```

```
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
      - podtemplates
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

本节中提供的角色定义只是一个示例。必须根据最终用户要求定义开发人员角色。

1. 同样，为 project-2 创建开发人员角色。
2. 所有 OpenShift 和 NetApp 存储资源通常由存储管理员管理。存储管理员的访问由安装 Trident 时创建的 Trident 操作员角色控制。此外，存储管理员还需要访问 ResourceQuotas 来控制存储的使用方式。
3. 在集群中的所有项目中创建一个用于管理 ResourceQuotas 的角色，以将其连接到存储管理员：

```
cat << EOF | oc create -f -
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: resource-quotas-role
rules:
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - resourcequotas
  - verbs:
      - '*'
    apiGroups:
      - quota.openshift.io
    resources:
      - '*'
EOF
```

4. 确保集群与组织的身份提供程序集成，并且用户组与集群组同步。以下示例显示身份提供程序已与集群集成并与用户组同步。

```
$ oc get groups
NAME                          USERS
ocp-netapp-storage-admins     ocp-netapp-storage-admin
ocp-project-1                 ocp-project-1-user
ocp-project-2                 ocp-project-2-user
```

1. 为存储管理员配置 ClusterRoleBindings 。

```
cat << EOF | oc create -f -
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-trident-operator
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: trident-operator
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-resource-quotas-cr
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: resource-quotas-role
EOF
```

对于存储管理员，必须绑定两个角色： Trident 操作员和资源配额。

1. 为开发人员创建 RoleBindings ，将开发人员项目 1 角色绑定到项目 1 中的相应组（ OCP-project-1 ）。

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-1-developer
  namespace: project-1
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-1
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-1
EOF
```

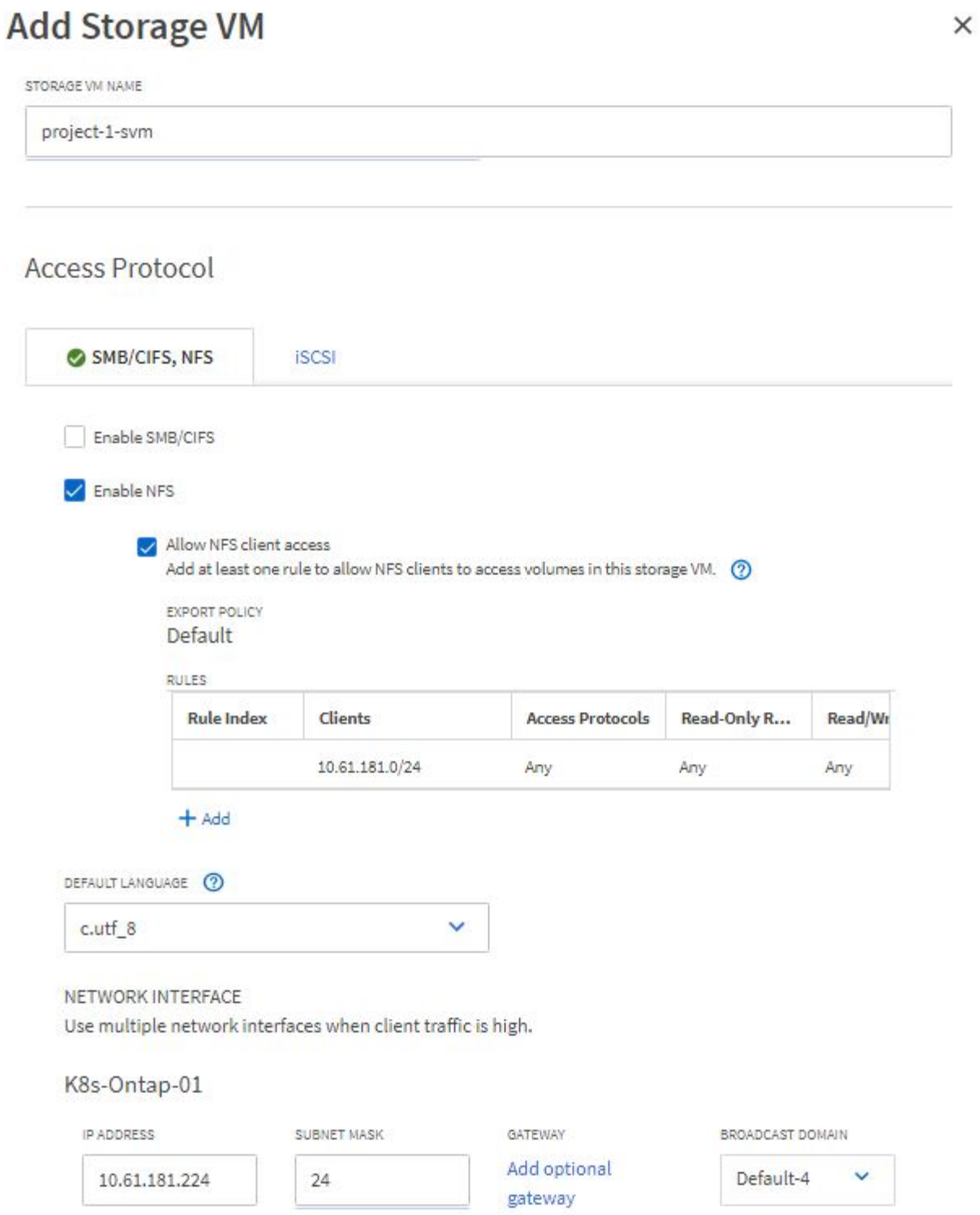
2. 同样，为开发人员创建 RoleBindings ，将开发人员角色绑定到 project-2 中的相应用户组。

"下一步：存储管理员任务。"

配置： **storage-admin** 任务

存储管理员必须配置以下资源：

1. 以管理员身份登录到 NetApp ONTAP 集群。
2. 导航到存储 > Storage VM ，然后单击添加。通过提供所需的详细信息，创建两个 SVM ，一个用于 project-1 ，另一个用于 project-2 。此外，还可以创建 vsadmin 帐户来管理 SVM 及其资源。



1. 以存储管理员身份登录到 Red Hat OpenShift 集群。
2. 为 project-1 创建后端，并将其映射到专用于该项目的 SVM 。NetApp 建议使用 SVM 的 vsadmin 帐户将后端连接到 SVM ，而不是使用 ONTAP 集群管理员。

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_1",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.224",
    "svm": "project-1-svm",
    "username": "vsadmin",
    "password": "NetApp123"
}
EOF
```

在此示例中，我们使用的是 ontap-NAS 驱动程序。根据使用情形创建后端时，请使用相应的驱动程序。

我们假定 Trident 已安装在 Trident 项目中。

1. 同样，为 project-2 创建 Trident 后端，并将其映射到专用于 project-2 的 SVM 。
2. 接下来，创建存储类。为 project-1 创建存储类，并通过设置 storagePools 参数将其配置为使用后端专用于 project-1 的存储池。

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-1-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_1:.*"
EOF
```

3. 同样，为 project-2 创建一个存储类，并将其配置为使用专用于 project-2 的后端存储池。
4. 创建 ResourceQuota 以限制 project-1 中的资源，从而从专用于其他项目的存储库请求存储。

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-1-sc-rq
  namespace: project-1
spec:
  hard:
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

5. 同样，也可以创建 ResourceQuota 来限制项目 2 中的资源，以便从专用于其他项目的存储库请求存储。

"下一步：验证。"

验证

要验证在上述步骤中配置的多租户架构，请完成以下步骤：

验证在分配的项目中创建 **PVC** 或 **Pod** 的访问权限

1. 以项目 1 中的 OCP-project-1-user 和开发人员身份登录。
2. 检查访问权限以创建新项目。

```
oc create ns sub-project-1
```

3. 使用分配给 project-1 的 storageclass 在 project-1 中创建 PVC 。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. 检查与 PVC 关联的 PV 。

```
oc get pv
```

5. 验证 PV 及其卷是否已在 NetApp ONTAP 上专用于 project-1 的 SVM 中创建。

```
volume show -vserver project-1-svm
```

6. 在 project-1 中创建 POD ，然后挂载上一步创建的 PVC 。

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  volumes:
    - name: test-pvc-project-1
      persistentVolumeClaim:
       claimName: test-pvc-project-1
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: test-pvc-project-1
EOF
```

7. 检查 POD 是否正在运行以及是否已挂载卷。

```
oc describe pods test-pvc-pod -n project-1
```

验证在其他项目中创建 **PVC** 或 **Pod** 的访问权限，或者使用专用于另一项目的资源

1. 以项目 1 中的 OCP-project-1-user 和开发人员身份登录。
2. 使用分配给 project-2 的 storageclass 在 project-1 中创建 PVC 。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1-sc-2
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-2-sc
EOF
```

3. 在 project-2 中创建 PVC 。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-2-sc-1
  namespace: project-2
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. 请确保未创建 PVC `test-vpa-project-1-sc-2` 和 `test-vpa-project-2-sc-1` 。

```
oc get pvc -n project-1
oc get pvc -n project-2
```

5. 在 project-2 中创建 POD 。

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
EOF
```

验证对查看和编辑项目， **ResourceQuotas** 和 **StorageClasses** 的访问权限

1. 以项目 1 中的 OCP-project-1-user 和开发人员身份登录。
2. 检查访问权限以创建新项目。

```
oc create ns sub-project-1
```

3. 验证对查看项目的访问权限。

```
oc get ns
```

4. 检查用户是否可以在 project-1 中查看或编辑 ResourceQuotas 。

```
oc get resourcequotas -n project-1
oc edit resourcequotas project-1-sc-rq -n project-1
```

5. 验证用户是否有权查看存储器。

```
oc get sc
```

6. 检查访问权限以描述存储器。
7. 验证用户的访问权限以编辑存储器库。

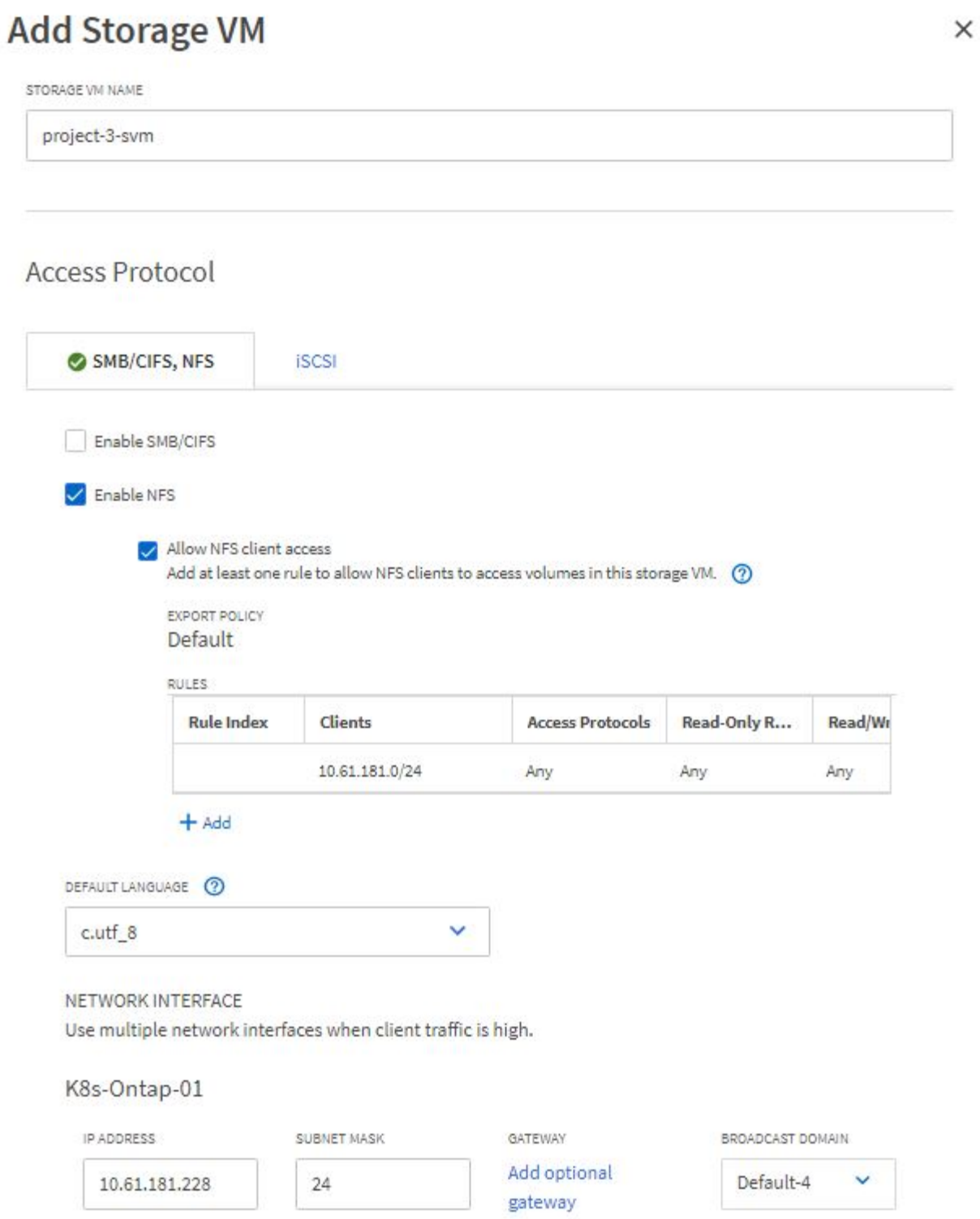
```
oc edit sc project-1-sc
```

"下一步：扩展。"

扩展：添加更多项目

在多租户配置中，使用存储资源添加新项目需要进行额外配置，以确保不会违反多租户要求。要在多租户集群中添加更多项目，请完成以下步骤：

1. 以存储管理员身份登录到 NetApp ONTAP 集群。
2. 导航到 `Storage → Storage VM`，然后单击 `Add`。创建一个专用于 project-3 的新 SVM 。此外，还可以创建 vsadmin 帐户来管理 SVM 及其资源。



1. 以集群管理员身份登录到 Red Hat OpenShift 集群。
2. 创建新项目。

```
oc create ns project-3
```

3. 确保已在 IdP 上为 project-3 创建用户组并与 OpenShift 集群同步。

```
oc get groups
```

4. 为 project-3 创建开发人员角色。

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-3
  name: developer-project-3
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
      - podtemplates
```

```
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

本节中提供的角色定义只是一个示例。必须根据最终用户要求定义开发人员角色。

1. 在 project-3 中为开发人员创建 RoleBinding. 将开发人员项目 3 角色绑定到 project-3 中的相应组（ OCP-project-3 ）。

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-3-developer
  namespace: project-3
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-3
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-3
EOF
```

2. 以存储管理员身份登录到 Red Hat OpenShift 集群
3. 创建 Trident 后端并将其映射到专用于 project-3 的 SVM 。NetApp 建议使用 SVM 的 vsadmin 帐户将后端连接到 SVM ，而不是使用 ONTAP 集群管理员。

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_3",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.228",
    "svm": "project-3-svm",
    "username": "vsadmin",
    "password": "NetApp!23"
}
EOF
```

在此示例中，我们使用的是 ontap-NAS 驱动程序。使用相应的驱动程序根据使用情形创建后端。

我们假定 Trident 已安装在 Trident 项目中。

1. 为 project-3 创建存储类，并将其配置为使用专用于 project-3 的后端存储池。

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-3-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_3:.*"
EOF
```

2. 创建 ResourceQuota 以限制项目 3 中的资源，从而从专用于其他项目的存储库请求存储。

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-3-sc-rq
  namespace: project-3
spec:
  hard:
    project-1-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

3. 在其他项目中修补 ResourceQuotas ，以限制这些项目中的资源从专用于项目 3 的存储库访问存储。

```
oc patch resourcequotas project-1-sc-rq -n project-1 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
oc patch resourcequotas project-2-sc-rq -n project-2 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
```

版权信息

版权所有©2022 NetApp、Inc.。保留所有权利。Printed in the U.S.版权所涵盖的本文档的任何部分不得以任何形式或任何手段复制、包括影印、录制、 磁带或存储在电子检索系统中—未经版权所有者事先书面许可。

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

本软件由NetApp按"原样"提供、不含任何明示或默示担保、包括但不限于适销性和特定用途适用性的默示担保、特此声明不承担任何责任。IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice.NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp.The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S.patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

商标信息

NetApp、NetApp标识和中列出的标记 http://www.netapp.com/TM 是NetApp、Inc.的商标。其他公司和产品名称可能是其各自所有者的商标。